Computer Organization and Assembly Language

Week 09 — Chapter 4: Bit Manipulations
Shifting, Rotation and the Multiplication Algorithm

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 09

Outline

1. Quick Review
2. Why This Chapter?
3. The Multiplication Algorithm
4. Shifting and Rotation
5. Multiplication in Assembly
6. Extended Operations

Quick Review: What Was Done Last Week?

- **Relative addressing** — a jump stores a **distance**, not an address. The CPU adds it to IP, which already points at the *next* instruction.

$$0103 + 0016 = 0119$$

- Because jumps store *gaps*, the whole program can be loaded anywhere and still work — **relocatable code**.
- **Three jump types:** short (2 bytes, ± 128), near (3 bytes, whole segment), far (5 bytes, absolute, loads CS and IP).
- Conditional jumps can only be short.
- **Bubble sort** — compare neighbours, swap if out of order, repeat until a pass makes no swap.
  - `JBE` not `JB` — swapping equal elements loops forever.
  - `cmp bx, 18` not 20 — 10 elements make 9 *pairs*.
- One instruction sets the whole behaviour: `JBE`/`JAE` unsigned, `JLE`/`JGE` signed.

Why Are We Studying Bit Manipulation?

The question that opens the chapter

The 8088 has no `MUL` we are allowed to use yet. So: **how do you multiply two numbers using only addition?**

- **Multiplication is the excuse, not the goal.** Working it out forces us to discover a whole family of instructions — **shifts and rotations** — that turn complicated tasks into two or three lines of code.
- **Shifting is how hardware multiplies and divides.** One shift left $= \times 2$; one shift right $= \div 2$. No multiplier circuit required.
- **Shifting is how you read individual bits.** Each shift drops one bit into the carry flag, where `JC` can test it.
- **Extended operations break the 16-bit barrier.** Two instructions together can shift, add and multiply numbers of *any* size.

The real lesson

Assembly gives you only basic building blocks. What you build from them is limited by your **imagination**, not by the instruction set.

Multiplication: the Method You Already Know

- Remember school: multiply by one digit, put a cross, multiply by the next digit, put two crosses, then add everything up.
- Familiar — but we never looked at it as an **algorithm**. To give it to a processor, we must.

$$
\begin{array}{rl}
1101 & = 13 \\
0101 & = 5 \\
\hline
1101 & \\
0000\mathrm{x} & \\
1101\mathrm{xx} & \\
0000\mathrm{xxx} & \\
\hline
01000001 & = 65
\end{array}
$$

- Note: the answer is **twice as long** as the operands — 4 bits $\times$ 4 bits gives 8 bits.

The Beauty of Base 2

No real multiplication is needed at the digit level

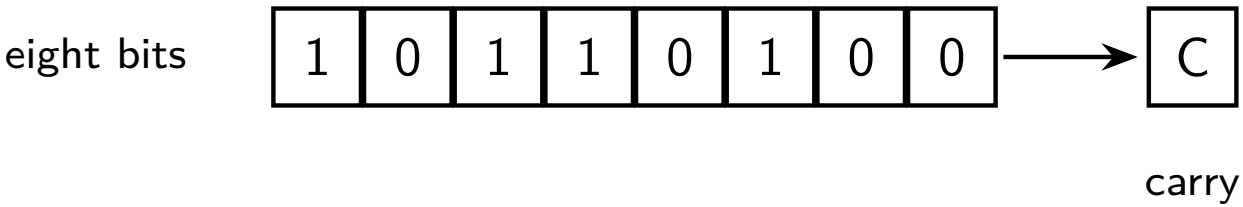
- Multiplier digit is **0** $\rightarrow$ the row is **0000**.
- Multiplier digit is **1** $\rightarrow$ the row is the **multiplicand itself**.

- In decimal you must actually know your tables. In binary there is nothing to compute — only **copy or don't copy**.
- The second observation: each row is written **one place further left** than the one above.
  - 1st digit — no shift
  - 2nd digit — shifted once
  - 3rd digit — shifted twice
  - 4th digit — shifted three times
- The crosses are just **placeholders** — treated as zeros in the final addition.

So multiplication = **testing bits** + **shifting** + **adding**. We have addition. We need the other two.

The Pipe-and-Balls Picture

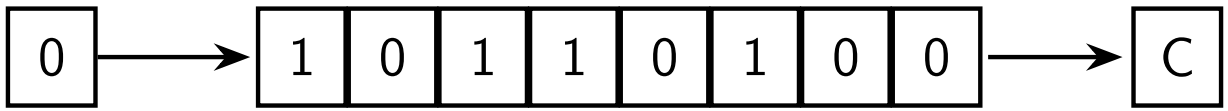
- Imagine a **pipe** holding exactly eight balls, open at both ends, with a **basket** (the carry flag) to catch whatever falls out.



- Push a ball in from one end and **every ball moves one place**; the ball forced out of the far end lands in the basket.
- In the diagrams the balls are drawn as **boxes holding the actual bit**, so you can follow the pattern 10110100 through every instruction.
- Sixteen-bit shifting works identically — just a longer pipe.
- Seven instructions, all built on this one picture: **SHR, SHL/SAL, SAR, ROR, ROL, RCR, RCL**.

Shift Logical Right — SHR

- A **zero** is inserted from the left; every bit moves one place right; the rightmost bit falls into the **carry**.

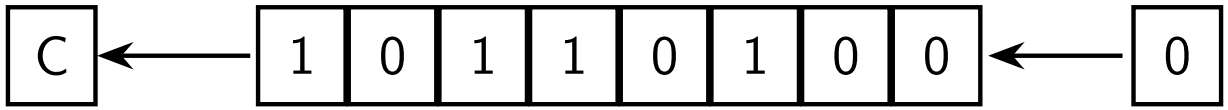


10110100	before	
01011010	after	CF = 0

SHR is **division by 2** — and the bit in CF is the **remainder**.

Shift Logical Left — SHL (also SAL)

- The exact opposite: a **zero** enters from the **right**, every bit moves left, and the **most significant bit** drops into the carry.
- SAL (shift arithmetic left) is just another **name** for the same instruction.

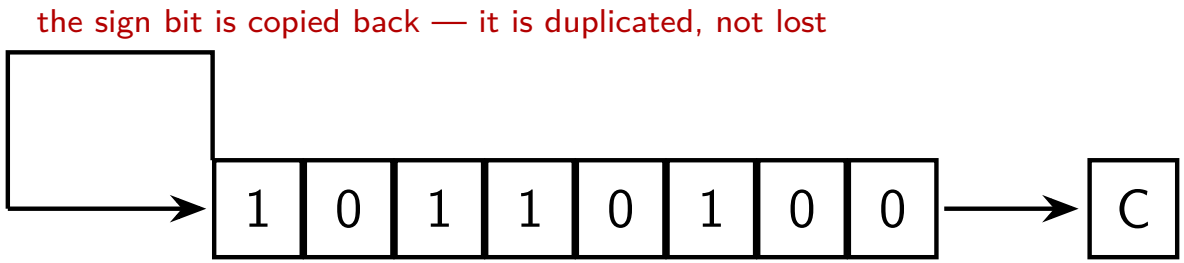


SHL is **multiplication by 2**.

- Careful: it is multiplication *only* if the bit dropped off the left was a zero. If a **1** is lost, the answer is wrong — the **overflow flag** signals this and can be tested with JO.

Shift Arithmetic Right — SAR

- A signed number keeps its **sign in the most significant bit**. A logical right shift would insert a zero there and change a negative number into a positive one.
- SAR shifts right but leaves a **copy of the sign bit** in place. The bit dropped on the right still goes to carry.

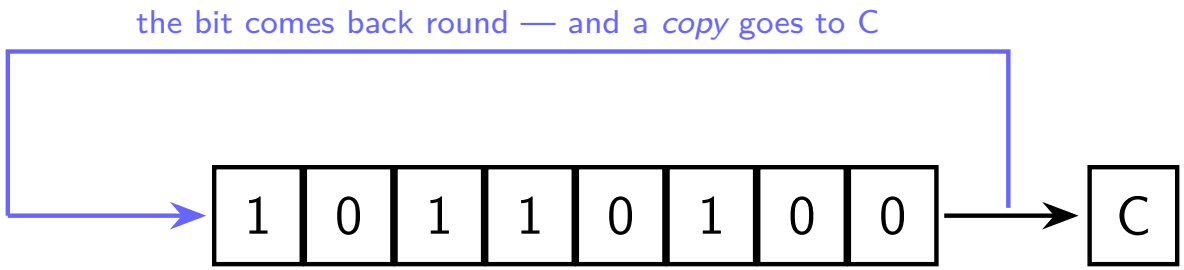


	unsigned ÷2	**signed ÷2**
use	SHR	SAR

- Using SHR on a negative number gives a wrong answer — the inserted zero destroys the sign.

Rotate Right — ROR

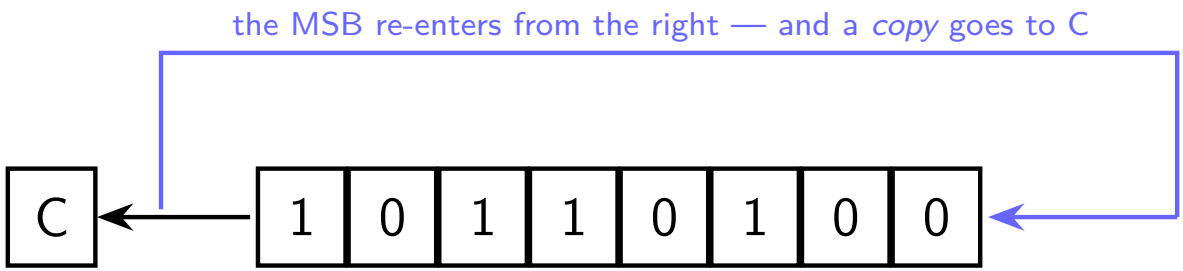
- Now **bend the pipe into a circle** so both ends meet.
- Every bit moves one place right; the bit leaving the right end **re-enters at the left**. A **copy** also goes to the carry flag.



- **Nothing is ever lost** in a rotation — the same eight bits simply change position.
- Rotation can be 8-bit or 16-bit, depending on the operand.

Rotate Left — ROL

- The reverse of ROR. The **most significant bit** is copied to the carry *and* re-inserted from the right, while every bit moves one place left.

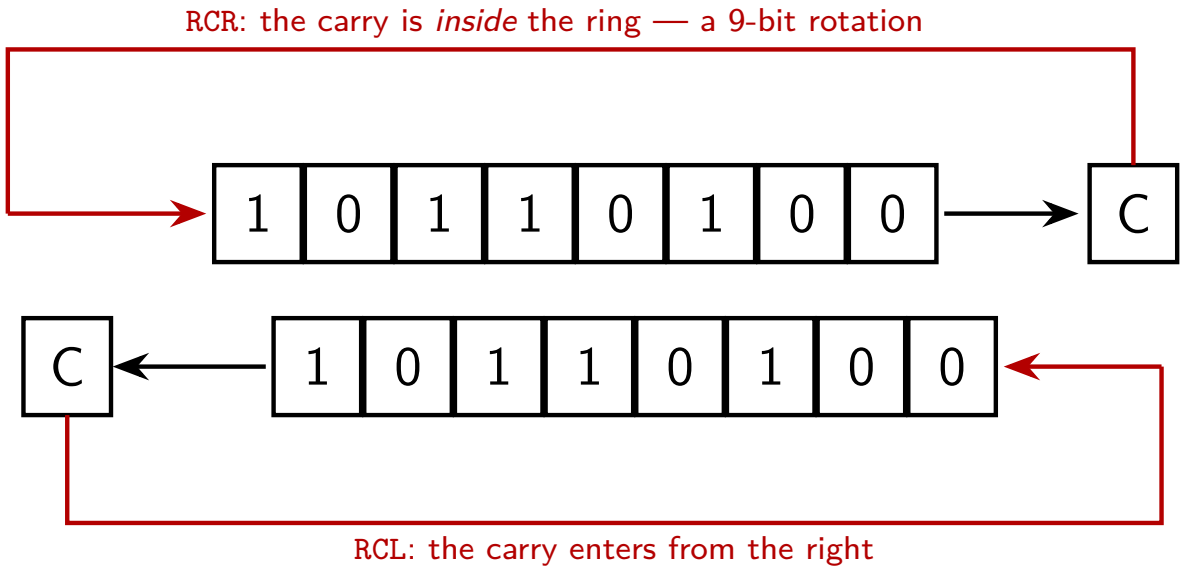


ROR/ROL vs SHR/SHL

A **shift** throws a bit away and brings in a zero. A **rotation** brings the same bit back round — the bit pattern is preserved, only reordered.

Rotate Through Carry — RCR and RCL

- Now put the **carry flag inside the circle**, between the two ends of the pipe.
- RCR: carry enters from the **left**; the rightmost bit becomes the new carry.
- RCL: carry enters from the **right**; the MSB becomes the new carry.



The crucial property

A **9-bit** (or **17-bit**) rotation — the carry is part of the ring. That extra bit is what lets us join two registers together, as we will see in extended shifting.

All Seven at a Glance

Instr.	What enters	What goes to CF	Use
`SHR`	0 from the left	rightmost bit	unsigned $\div 2$; read the low bit
`SHL` / `SAL`	0 from the right	leftmost bit	$\times 2$ (watch OF)
`SAR`	a copy of the sign bit	rightmost bit	**signed** $\div 2$
`ROR`	the bit that left the right	a copy of that bit	reorder, lose nothing
`ROL`	the bit that left the left	a copy of that bit	reorder, lose nothing
`RCR`	the **carry flag**	rightmost bit	join registers, shift right
`RCL`	the **carry flag**	leftmost bit	join registers, shift left

- **Shift** $\rightarrow$ a bit is lost. **Rotate** $\rightarrow$ nothing is lost. **Rotate through carry** $\rightarrow$ the carry joins the ring.

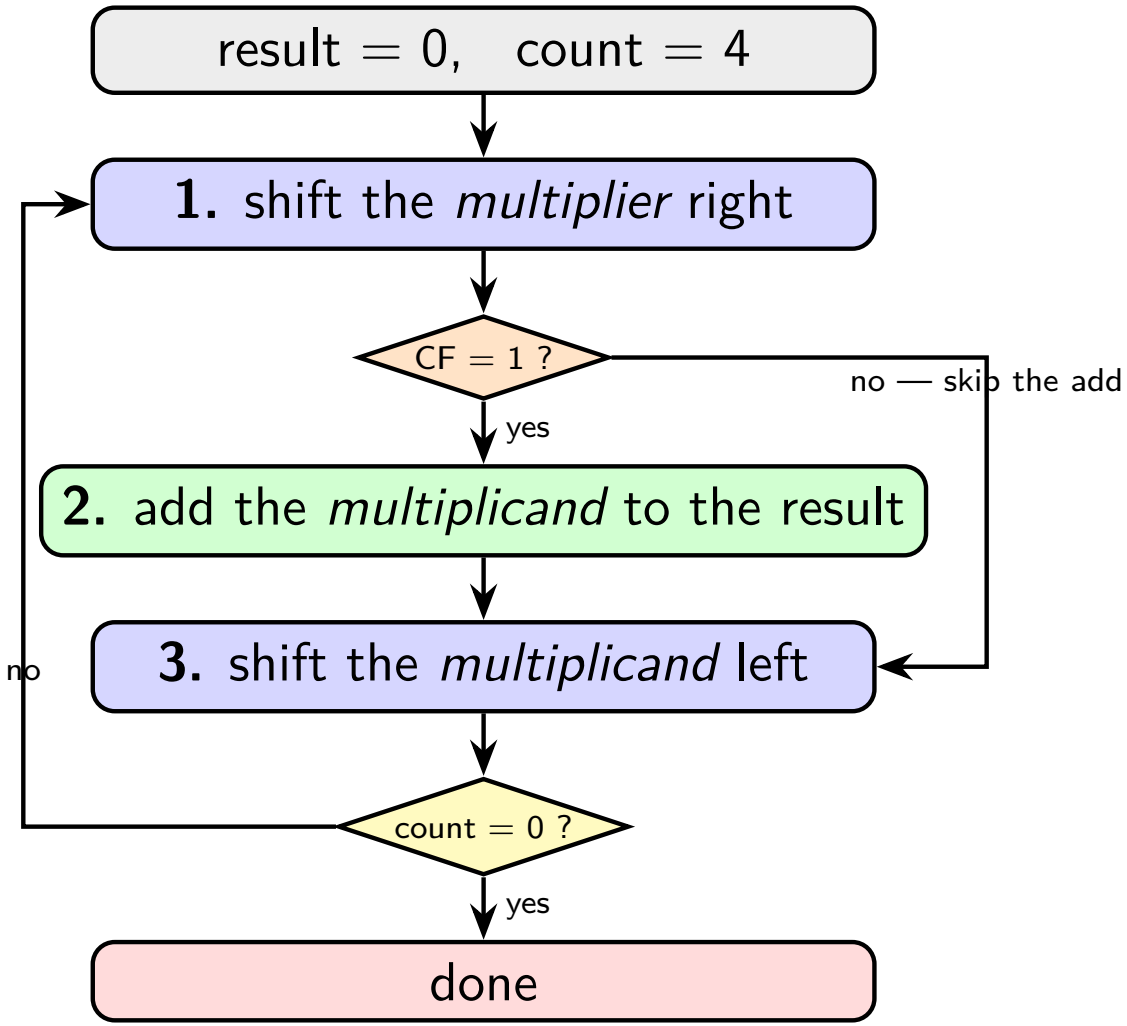
One Improvement Before We Code It

- The school method computes *all* the rows first, then adds them. That means remembering many intermediate answers.
- **Better:** add each row into a running total as soon as it is produced. Same answer, nothing to remember.

	Accumulated Result
	0 (initial)
`1101 = 13`	$0 + 13 = 13$
`0000x = 0`	$13 + 0 = 13$
`1101xx = 52`	$13 + 52 = 65$
`0000xxx = 0`	$\mathbf{65 + 0 = 65}$

This is the version a processor can execute — one small step at a time, with only **one** number to keep.

The Algorithm in Four Steps



- **4.** Repeat once for every bit of the multiplier — 4 times here, 8 for byte operands, 16 for word operands.

Two Design Details Worth Noticing

1. How do we test a bit? We shift it into the carry.

`SHR` drops the rightmost bit into CF. So `shr dl, 1` followed by `jnc` is **testing the low bit of the multiplier**. The next shift brings the next bit down, and so on — one bit per iteration, automatically.

2. Why the shifts sit where they do

- The multiplicand is shifted **after** the addition — because in the school method the first row has *no* cross. No shift on the first pass.
- The multiplier is shifted **before** the test — because the test depends on the bit that the shift produces.

- The multiplicand is shifted every iteration, whatever the multiplier bit was — exactly like the crosses, which are written whether or not the digit was zero.

Example 4.1 — 4-bit Multiplication

```
; 4-bit multiplication algorithm
[org 0x0100]
            jmp start

multiplicand: db 13 ; 4-bit multiplicand (8-bit space)
multiplier: db 5 ; 4-bit multiplier
result: db 0 ; 8-bit result

start: mov cl, 4 ; initialize bit count to four
         mov bl, [multiplicand] ; load multiplicand in bl
         mov dl, [multiplier] ; load multiplier in dl

checkbit: shr dl, 1 ; move right most bit in carry
         jnc skip ; skip addition if bit is zero
         add [result], bl ; accumulate result
skip: shl bl, 1 ; shift multiplicand left
         dec cl ; decrement bit count
         jnz checkbit ; repeat if bits left

         mov ax, 0x4c00
         int 0x21
```

- DEC is new — it simply subtracts one from its single operand, and sets the flags.

Example 4.1 — Traced, 13×5

Iter	DL after `shr`	CF	add?	result	BL after `shl`
start	0101 = 5	—	—	0	1101 = 13
1	0010 = 2	**1**	+13	**13**	26
2	0001 = 1	0	skip	13	52
3	0000 = 0	**1**	+52	**65**	104
4	0000 = 0	0	skip	**65**	208

- `CL` counts $4 \to 3 \to 2 \to 1 \to 0$; when it reaches zero, `JNZ` falls through and the program ends.
- **Result = 65** — stored in `[result]`.
- In the debugger: `SHR` is dividing DL by 2 with the remainder landing in CF; `SHL` is doubling BL so that it is added one place further left each time.

Why 16-bit Operations Are Not Enough

- A 4-bit example explains the algorithm — but the point of a computer is to multiply **large** numbers.
- 16×16 bits gives a **32-bit** answer. We would need to shift and add 32-bit values, and the 8088 only does 16 bits at a time.

What goes wrong — a concrete example

Take $40000 = 9C40_{16} = 1001110001000000_2$. Shift it left once to double it:

$$1001110001000000 \rightarrow 0011100010000000$$

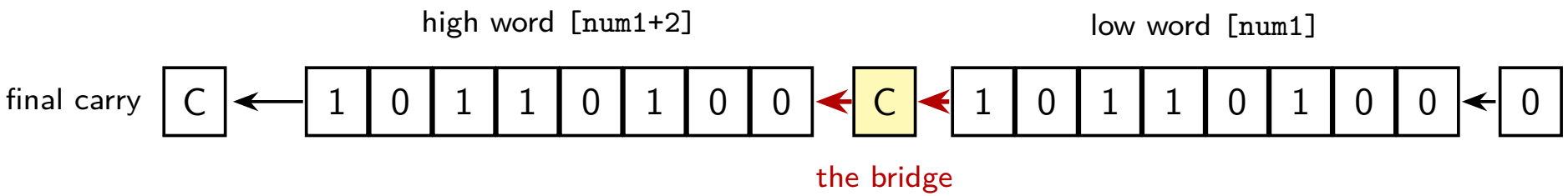
The leading **1** fell into the carry and was lost. We get $3880_{16} = 14464$ instead of the correct 17-bit answer $13880_{16} = 80000$.

- **The fix:** store the 16-bit number in **32-bit space** with the upper half zeroed, then shift all 32 bits as a unit — so no significant bit can ever fall off.

Extended Shifting — Left

```
num1: dd 40000 ; 32 bits: low word at num1, high at num1+2

      shl word [num1], 1 ; shift low half; MSB falls into CF
      rcl word [num1+2], 1 ; CF is pushed into the LSB of the high half
```



- The two instructions are carefully chosen so the carry flag acts as a **bridge** joining two 16-bit words into one 32-bit number.
- **Not limited to 32 bits** — repeat the `RCL` for as many words as you like. 64-bit, 1024-bit: the only real limit is memory.

Extended Shifting — Right

```
num1: dd 40000

      shr word [num1+2], 1 ; HIGH half first!
      rcr word [num1], 1 ; CF pushed into the MSB of the low half
```

Order matters — and it is the opposite way round

Shifting **left**: low half first, then the high half.
Shifting **right**: **high half first**, then the low half.

In each case the half that *loses* a bit must go first, so the carry is ready when the other half needs it.

- For a **signed** number, use `SAR` instead of `SHR` on the high half — so the sign is preserved.
- The low half still uses `RCR`: it is receiving a bit, not a sign.

Extended Addition — the ADC Instruction

ADC — "add with carry"

Normal ADD has two operands. ADC has a **third, implied** operand: the **carry flag**.

ADC AX, BX $\Rightarrow$ AX = AX + BX + CF

```
dest: dd 40000
src: dd 80000

     mov ax, [src]
     add word [dest], ax ; low halves: NORMAL add
     mov ax, [src+2]
     adc word [dest+2], ax ; high halves: add WITH carry
```

- If the low halves produced a carry, it **must** reach the high halves. A plain ADD would lose it and the answer would be wrong.
- One operand must be in a register, so AX ferries each half of the source; the destination is updated in place.
- The final carry stays in CF — test it with JC to detect overflow of the whole 32-bit result.

Extended Subtraction — SBB

```
dest: dd 40000
src: dd 80000

      mov ax, [src]
      sub word [dest], ax ; low halves: NORMAL subtract
      mov ax, [src+2]
      sbb word [dest+2], ax ; high halves: subtract WITH borrow
```

- SBB = "subtract with borrow". If the low halves needed a borrow, **one extra is subtracted** from the high halves.
- **Why "borrow" and not "carry"?** It is the same flag — CF holds the carry for additions and the borrow for subtractions. Both are generated at the 17th bit, and no single instruction ever needs both meanings at once.

	low half	**high half**
addition	ADD	ADC
subtraction	SUB	SBB

Example 4.2 — 16-bit Multiplication

```
; 16-bit multiplication
[org 0x0100]
            jmp start

multiplicand: dd 1300 ; 16-bit multiplicand in 32-bit space
multiplier: dw 500 ; 16-bit multiplier
result: dd 0 ; 32-bit result

start: mov cl, 16 ; initialize bit count to 16
         mov dx, [multiplier] ; load multiplier in dx

checkbit: shr dx, 1 ; move right most bit in carry
         jnc skip ; skip addition if bit is zero
         mov ax, [multiplicand]
         add [result], ax ; add less significant word
         mov ax, [multiplicand+2]
         adc [result+2], ax ; add more significant word

skip: shl word [multiplicand], 1
         rcl word [multiplicand+2], 1 ; shift multiplicand left (32-bit)
         dec cl ; decrement bit count
         jnz checkbit ; repeat if bits left

         mov ax, 0x4c00
         int 0x21
```

Example 4.2 — What Changed, and Why

	Example 4.1 (4-bit)	**Example 4.2 (16-bit)**
multiplier	`db` — shifted in DL	`dw` — shifted in DX *(still only 16 bits: we just read its bits)*
multiplicand	`db` — 8-bit space	`dd` — **32-bit space**, so 16 left shifts lose nothing
addition	`add [result], bl`	`add` + `adc` — extended 32-bit
shifting	`shl bl, 1`	`shl` + `rcl` — extended 32-bit
count	4	16

- **The algorithm is identical.** Only the *width* of the operations changed.
- The multiplier stays 16-bit because we only ever read its bits one at a time — it is never shifted left.

Example 4.2 — Inside the Debugger

Variable	Offsets	Size
`multiplicand`	0103–0106	4 bytes (`dd`)
`multiplier`	0107–0108	2 bytes (`dw`)
`result`	0109–010C	4 bytes (`dd`)

- At the end, `result` holds $\texttt{0009EB10}_{16} = 650{,}000$ — and indeed $1300 \times 500 = 650{,}000$. ✓
- Watch the multiplicand: it starts as `00000514` ($= 1300$) and finishes as `05140000` after 16 left shifts.
- That is exactly what a true 32-bit shift would have produced — **two instructions did the work of an instruction the CPU does not have**.

Extended shifting and extended addition give the *same effect* as 32-bit instructions, built entirely from 16-bit ones.

Summary

- Binary multiplication needs no multiplication table: bit $= 0 \rightarrow$ add nothing; bit $= 1 \rightarrow$ add the multiplicand. Each row shifts one place left.
- **Shifts:** `SHR` ($\div 2$), `SHL` ($\times 2$), `SAR` (signed $\div 2$, keeps the sign bit).
- **Rotations:** `ROR`, `ROL` lose nothing; `RCR`, `RCL` put the carry *inside* the ring — a 9- or 17-bit rotation.
- **Testing a bit** = shifting it into CF and using `JC`/`JNC`.
- **The multiply loop:** shift the multiplier right; if CF=1 add the multiplicand; shift the multiplicand left; repeat once per bit.
- **Extended operations** break the 16-bit limit:
  - shift left: `SHL` low, then `RCL` high
  - shift right: `SHR` high, then `RCR` low
  - add: `ADD` low, then `ADC` high
  - subtract: `SUB` low, then `SBB` high
- A 16-bit number that will be shifted left must live in **32-bit space**, or significant bits fall off.

Thank You

Any Questions?

"Assembly language only gives you the basic building blocks. Whether you build a hut or a palace from them depends only on your imagination."

Next: division, logical instructions and bitwise masking.